Встреча 10

Разработка микросервисных приложений на Golang

YADRO

Часто встречающиеся ошибки и вопросы

- Как эмулировать ошибки?
- Как тестировать БД?

Задание 10 - Freestyle



Задание 10 - Freestyle

- HTML gateway: сервер-адаптер HTML <-> REST API

Задание 10 - Freestyle

- HTML gateway
- Telegram bot

Задание 10 - Freestyle

- HTML gateway
- Telegram bot
- Собственная имплементация
    - синглетон, например 😂
    - MQ вместо gRPC,
    - MySQL / MongoDB / Redis, …

Задание 10 - Freestyle

- HTML gateway
- Telegram bot
- Собственная имплементация
    - синглетон, например 😂
    - MQ вместо gRPC,
    - MySQL / MongoDB / Redis, …
    - Android/iOS app
- Multi-instance microservices - minikube
- …

Задание 10 - Freestyle

- HTML gateway
- Telegram bot
- Собственная имплементация
    - синглетон, например 😂
    - MQ вместо gRPC,
    - MySQL / MongoDB / Redis, …
    - Android/iOS app
- Multi-instance microservices - minikube
- …

+ 5-минутный ролик-демо

Чистый (стильный, классный) код

“код, который написан с учетом его будущей эволюции, легко читаемый и понятный другим разработчикам”

- Алиса

Чистый (стильный, классный) код

- Простота
- Самодокументированность
- Согласованность
- Читаемость

Инструментарий

- golangci-lint

Инструментарий

- golangci-lint
- gocyclo

```
1 is the base complexity of a function

+1 for each 'if', 'for', 'case', '&&' or '||'
```

Инструментарий

- golangci-lint
- gocyclo
- gocognit

https://www.sonarsource.com/docs/CognitiveComplexity.pdf

Cyclo vs Cognit

```
func SumOfPrimes(max int) int {      // +1
    var total int

OUT:
    for i := 1; i < max; i++ {       // +1
        for j := 2; j < i; j++ {     // +1
            if i%j == 0 {            // +1
                continue OUT
            }
        }
        total += i
    }

    return total
} // Cyclomatic complexity = 4
```

Cyclo vs Cognit

```
func SumOfPrimes(max int) int {
    var total int

OUT:
    for i := 1; i < max; i++ {          // +1
        for j := 2; j < i; j++ {        // +2 (nesting = 1)
            if i%j == 0 {               // +3 (nesting = 2)
                continue OUT            // +1
            }
        }
        total += i
    }

    return total
} // Cognitive complexity = 7
```

Идиоматичный код - переменные

```
var defaultZeroVar int

nonZeroVar := 123
```

Идиоматичный код - константы

```
const (
    xkcdParallelFetch = 100
    myPi = 3.14 // untyped, can be both float32 / float64
)
```

Идиоматичный код - константы

```
const (
    xkcdParallelFetch = 100
    myPi = 3.14 // untyped, can be both float32 / float64
)
```

неиспользуемые константы не удаляются,
держим как можно ближе к потребителю

Идиоматичный код - happy path, error path

```
func divide(a, b int) (int, err) {
    if b == 0 {
        return 0, errors.New(“divide by zero”)
    }
    // happy path unindented
```

functions vs interfaces

```
func calculate(
    a, b int,
    add func(int, int) int,
) int {
    …
}
```

```
func calculate(
    a, b int,
    add Adder,
) int {
    …
}
```

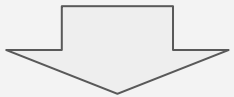
Принимаем интерфейсы, возвращаем структуры

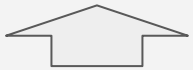
```
type (
    Text struct {
        content []byte
        language string
    }
    Formatter interface {
        Format(string) string
    }

func process(t string, l language, f Formatter) (Text, err) {
    ...
```

Г - гибкость

любой форматировщик





любой клиент

Два типа объектов

- Объект-сервис
  - выполняют задачу
  - предоставляют информацию
  - ничего не меняют внутри себя
  - создаются один раз (и навсегда)
- Объект-значение (данные)
  - материал для сервисов
  - ограниченный жизненный цикл

Controller, Processor, …

User, Product, Report …

Зависимости

через

- конструктор (func New(...)) 🤗
- параметр метода 🤗
- создаем сами 😕
- глобальный локатор 😕

Зависимости

явное и простое лучше, чем сложное и скрытое

```
func New(c Config) DB, error {
    db, err := sqlx.Connect(
        “postgres”,
        c.Search.DB.ConnectStr,
    )
```

```
func New(dsn string) DB, error {
    db, err := sqlx.Connect(
        “postgres,
        dsn,
    )
```

Объекты-значения

- Нет зависимостей, только значения
- Проверка данных - в конструкторах
- Сеттеры - зло (нужно проверять инварианты, как в конструкторе)
- Исключение - DTO

Сущности

- Изменяемые объекты
- Идентифицируются
- Методы-модификаторы - глаголы
  - в неизменяемых объектах - существительные

```
func (l Logger) WithDebug() Logger // idiomatic - *Logger

func (l *Logger) SetDebug()
```

x.Increment() vs x.Incremented()

Command Query Separation (CQS)

- Запросы - не изменяют состояние, возвращают значения
- Команды - изменяют состояния, возвращают только ошибку

```
func (c *Counter) Increment() error

func (c  Counter) Get() int // idiomatic way - *Counter
```

- CQRS - Command Query Responsibility Segregation, развитие CQS

Command Query Separation (CQS)

Плюсы

- простота
- производительность
- масштабируемость
- безопасность

Минусы

- Атомарность
- Многопоточность

Текучие интерфейсы - для неизменяемых объектов?

```
qb := querybulder.New()
query1 := qb.Select(“a”).From(“c”).Where(“e”).Limit(“g”)
query2 := qb.Select(“b”).From(“d”).Where(“f”).Limit(“h”)
query2 == query1 ????
```